

A Membership Inference and Adversarial Attack Defense Framework for Network Traffic Classifiers

Guangrui Liu , Weizhe Zhang , Senior Member, IEEE, Xurun Wang , Stephen King ,
and Shui Yu , Fellow, IEEE

Abstract—Malicious traffic identification methods in intrusion detection systems have evolved from rule-based matching to machine learning. However, security risks such as membership inference and adversarial attacks hinder the practical deployment of machine learning-based network intrusion detection systems (ML-NIDSs). In this work, we design a defense framework called hierarchical differential privacy (HierarchicalDP) to safeguard ML-NIDS against membership inference and adversarial attacks. First, we analyze the principles of membership inference and adversarial attacks to find their correlation. Based on this, we propose the feature distribution security metric (FDSM) to measure the risk of membership inference and adversarial attacks on ML-NIDS. Then, we design the HierarchicalDP framework, which partitions network traffic sample features according to security levels and introduces distinct noise on each security level feature to satisfy FDSM, thus defending against membership inference and adversarial attacks. Finally, we evaluate the defensive performance of the HierarchicalDP framework on two network traffic datasets and four machine-learning models. The HierarchicalDP defense framework, based on Laplace noise, reduces the success rate of membership inference from 64.9% to 54.4% (ineffective binary classification), the evasion rate of adversarial samples from 86.1% to 23.2%, and maintains model accuracy (ACC) fluctuations within 4.2%. Furthermore, the HierarchicalDP framework adjusts sample features without modifying the model, thereby not affecting the inference speed. HierarchicalDP offers efficient and convenient defenses for ML-NIDS deployed in a network.

Impact Statement—Artificial intelligence (AI)-driven network intrusion detection is on the rise. ML-NIDSs are superior to rule-based NIDS in identifying new network attacks. However, AI security problems hinder ML-NIDS deployment. Existing

AI security defenses ignore the network traffic characteristics, resulting in weak protection for ML-NIDS. They not only failed to effectively defend against member inference and adversarial attacks, but they also lead to a decrease in ACC. This article introduces the HierarchicalDP defense framework to protect ML-NIDS. HierarchicalDP reduces the success rate of membership inference to 54.4% (ineffective binary classification), lowers the evasion rate of adversarial samples to 23.2%, and maintains ACC fluctuations within 4.2%. Furthermore, HierarchicalDP does not affect the inference speed, providing efficient and convenient defenses for ML-NIDS.

Index Terms—Artificial intelligence (AI) in cybersecurity, artificial intelligence safety, machine learning, responsible artificial intelligence, verifiable artificial intelligence.

I. INTRODUCTION

THE number of security vendors planning to integrate artificial intelligence (AI) technology into network intrusion detection systems is increasing [1]. However, the practical deployment of machine learning-based network intrusion detection systems (ML-NIDSs) often encounters obstacles [2]. On the one hand, the accuracy of malicious traffic identification by ML-NIDS is unstable. On the other hand, the machine learning security risks make ML-NIDS susceptible to attacks [3].

Membership inference and adversarial attacks are two primary machine learning security risks [4], [5]. Membership inference threatens the privacy of a model. This attack primarily stems from vulnerabilities in the model training process, where slight differences between training samples can be amplified and reflected in the training results of the model. Adversarial attacks threaten the inference accuracy of a model. This attack primarily stems from vulnerabilities in the model inference process, where specific small perturbations to test samples can be amplified and reflected in the inference results of the model.

Most existing defenses against membership inference and adversarial attacks are designed for computer vision [6], [7], which neglects particular characteristics of network traffic. When applied to network traffic classifiers, these defenses not only exhibit poor protection performance but also yield a significant drop in ACC [8], [9]. To address this issue, we introduce a defense framework custom-built to protect ML-NIDS called HierarchicalDP. This framework leverages the structural characteristics of network traffic samples. The HierarchicalDP framework adjusts sample features without modifying the model, thereby not affecting the inference speed. HierarchicalDP offers efficient and convenient defenses for ML-NIDS

Manuscript received 30 October 2023; revised 18 December 2023; accepted 19 January 2024. Date of publication 26 January 2024; date of current version 27 February 2025. This work was supported in part by the Joint Funds of the National Natural Science Foundation of China under Grant U22A2036 and in part by the National Key Research and Development Program of China under Grant 2021YFB3101102. This article was recommended for publication by Associate Editor Qinghua Lu upon evaluation of the reviewers' comments. (Corresponding author: Guangrui Liu.)

Guangrui Liu and Xurun Wang are with the School of Cyberspace Science, Harbin Institute of Technology, Harbin, Heilongjiang 150001, China (e-mail: liuguangrui@hit.edu.cn; 22S003113@stu.hit.edu.cn).

Weizhe Zhang is with the School of Cyberspace Science, Harbin Institute of Technology, Harbin, Heilongjiang 150001, China, and also with Peng Cheng Laboratory, Shenzhen, Guangdong 518055, China (e-mail: wzzhang@hit.edu.cn).

Stephen King is with Leland High School, San Jose, CA 95120 USA (e-mail: stephenking192008@gmail.com).

Shui Yu is with the School of Computer Science, University of Technology Sydney, Ultimo, NSW 2007, Australia (e-mail: shui.yu@uts.edu.au).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TAI.2024.3357791>, provided by the authors.

Digital Object Identifier 10.1109/TAI.2024.3357791

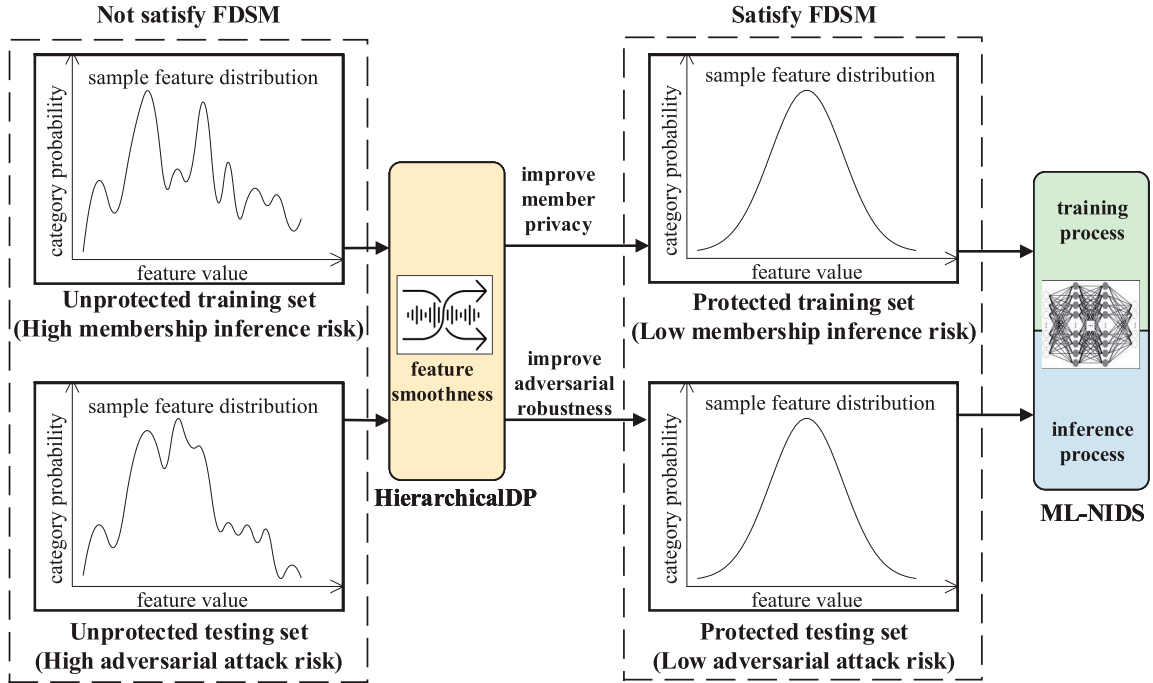


Fig. 1. Hierarchical differential privacy (HierarchicalDP) defense effectiveness and FDSM operation principle.

deployed in a network. There are three major challenges in designing this new defense framework.

- 1) *CH1: How to design a security metric method for network traffic classifiers?* For member inference risk, existing methods [10] use differential privacy to measure the privacy security of the model, but they lack consideration for the characteristics of network traffic samples. For adversarial attack risk, existing methods [11] conduct adversarial robustness testing after model training is finished, lacking a way to evaluate the adversarial attack risk before model training.
- 2) *CH2: How to identify the security-sensitive features of network traffic samples?* Network traffic samples exhibit structured characteristics, with the feature distributions of samples in the same category displaying similarity [12]. Moreover, the impact of each feature on model privacy and robustness varies. If privacy- and category-sensitive features can be identified and protected, it can minimize the impact of defense methods on model performance. This aspect is rarely considered in existing work.
- 3) *CH3: How to design differential privacy for ML-NIDS to defend against member inference and adversarial attacks?* On the one hand, since most ML-NIDS use incremental learning to update models [1], the defenses should not change the model structure but rather add noise to sample features. On the other hand, different sensitivity features should add different noise to effectively protect the security of ML-NIDS while keeping the added noise as small as possible.

To address the above challenges, we design the HierarchicalDP, as shown in Fig. 1, which solves the above problems

by feature smoothing based on the structured characteristics of traffic samples. First, since differential privacy and adversarial robustness are quantifiable metrics for describing membership inference and adversarial attack risks, we analyze their correlation and combine them by defining the feature distribution security metric (FDSM). FDSM aims to ensure that the feature probability distribution of samples meets the robustness requirement under the security budget. Then, we design the HierarchicalDP defense framework based on FDSM, which realizes the smoothing of feature distribution by adding differential noise to the critical features of the samples. The critical features contain privacy- and category-sensitive features computed by the feature hierarchization strategy based on the attention mechanism. HierarchicalDP can defend ML-NIDS based on raw traffic or statistical feature datasets. Due to only modifying input data, it applies to any commonly used machine learning model.

The contributions of our work are summarized as follows.

- 1) Targeting *CH1*, we introduce the FDSM. By extending the existing descriptions of differential privacy and adversarial robustness using Rényi divergence, we unify their formal expressions under α -metric and β -divergence.
- 2) Targeting *CH2*, we calculate the contribution of each feature to the predicted label using the self-attention mechanism. Then, we design a feature hierarchization strategy to identify the privacy- and category-sensitive features of network traffic samples.
- 3) Targeting *CH3*, we design HierarchicalDP defense framework for ML-NIDS. For training samples, we add differential noise to privacy-sensitive features, enabling a lightweight differential privacy mechanism. For test

samples, we introduce differential perturbations to category-sensitive features, achieving provably secure techniques for enhancing adversarial robustness.

- 4) Furthermore, we evaluate the defense effectiveness of HierarchicalDP on two publicly available malicious traffic datasets, consisting of packet and session samples, using four standard models. Compared to the state-of-the-art existing methods, HierarchicalDP reduces the membership inference success rate (MIR) by 8.4% and lowers the adversarial example escape rate (AER) by 23.7% while keeping the impact on ACC within 4.2%.

The rest is organized as below. Related works are discussed in Section II. The analysis of the correlation between member privacy and adversarial robustness and the formal definition of FDSM are introduced in Section III. The feature hierarchization strategy and HierarchicalDP algorithms are explained in Section IV. Experiment results are shown and discussed in Section V. Finally, we conclude our work in Section VI.

II. RELATED WORKS

In this section, we review related works in machine learning on differential privacy and adversarial robustness, as well as related works on the attention mechanism, an essential technique for HierarchicalDP to determine key sample features.

A. Differential Privacy

Differential privacy was proposed by Dwork et al. [13], which uses randomized responses to ensure that the database output is affected by a single record below a threshold. It became the gold standard for data security due to its rigorous quantitative proof of privacy leakage. Chatzikokolakis et al. [14] extended the measure of dataset variation from Hamming distance to an arbitrary metric space. Mironov [15] generalized the extent of the difference in statistical information of datasets from Max-divergence to Rényi divergence.

The differential privacy mechanism for machine learning models [10] is divided into adding random noise to the output, gradient, loss function, and input. Adding noise to the output and loss function significantly affects the ACC. Differential privacy can be achieved by adding a small amount of noise to the gradient with the help of composition theory, but strict differential privacy cannot be achieved. Studies on adding noise to the input have focused on distributed problems related to local differential privacy.

B. Adversarial Robustness

Adversarial example was discovered by Szegedy et al. [16], which enables a machine learning model to output a high confidence misclassification result by adding a specific small perturbation to the sample. Goodfellow et al. [17] formally described adversarial examples and improved model adversarial robustness through adversarial training. The first concise definition of adversarial robustness was given by Diochnos et al. [18]. Then, Pinot et al. [19] further analyzed the relationship between adversarial robustness and differential privacy.

TABLE I
IMPORTANT NOTATIONS

Symbol	Definition
$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id})$	The vector of sample features
$\mathbf{f}_i = (f_{i1}, f_{i2}, \dots, f_{id})$	The distribution of sample features
$i (1 \leq i \leq n)$	The index of samples
$j (1 \leq j \leq d)$	The index of feature dimensions
$\mathcal{Y} = \{Y_1, Y_2, \dots, Y_m\}$	The set of sample category labels
$c (1 \leq c \leq m)$	The index of category labels
$\mathbf{w} \sim Y_c = (w_1, w_2, \dots, w_d)$	The feature weights of category Y_c
S_{train}	The set of training samples
S_{test}	The set of testing samples
$\mathcal{X}$	The set of all possible samples
F	The model of machine learning
α	The norm of the metric
β	The order of divergence

Various studies enhance the adversarial robustness of machine learning models, but they interpret the adversarial example differently. Standard robustness enhancement techniques [11] include adversarial training, feature denoising, and random transformation. However, most existing works only test the effectiveness of methods experimentally and rarely theoretically analyze the adversarial robustness. Diochnos et al. [18] gave a more reasonable formal description of adversarial robustness, but there is room for generalization.

C. Attention Mechanism

The attention mechanism was proposed by Bahdanau et al. [20] to improve the effectiveness of models in processing sequential information. The core idea of the attention mechanism is the kernel regression theory proposed by Nadaraya and Watson in statistics. The self-attention mechanism allows for analyzing the correlations among the internal elements in a single sequence. The multiheaded self-attention (MSA) proposed by Vaswani et al. [21] improves the accuracy of feature correlation by taking the average of multiple correlation matrices. Since attention scores and weights reflect the impact of different features on the classification results of machine learning models, many works use the attention mechanism to estimate and rank the contribution of sample features to category labels [22].

III. FEATURE DISTRIBUTION SECURITY METRIC

In this section, we analyze the characteristics of network traffic samples. Then, we propose a FDSM for ML-NIDS, which unifies the definitions of differential privacy and adversarial robustness. Table I lists the important notations used in this article.

A. Network Traffic Characteristics

Previous studies have analyzed the relationship between differential privacy and adversarial robustness in machine learning [23]. However, due to the lack of the premise assumption with structured samples, providing a unified description of both aspects is challenging. Furthermore, existing research either confuses the machine learning training and inference processes [19]

or only examines one of the scenarios [24]. In this section, our contribution lies in systematically exploring these relationships and articulating their practical significance.

Two data preprocessing methods exist for network traffic samples in ML-NIDSs [12].

- 1) *Packet samples* (network traffic samples based on packet field information): Each packet is an independent sample, and the field information of each packet is recorded.
- 2) *Session samples* (network traffic samples based on session statistics information): Packets are grouped into sessions, and each session is a sample, with the statistical information of the session being recorded.

Both types of samples exhibit structured characteristics, with the feature distributions of samples in the same category displaying similarity. This is because traffic sessions with the same network attack have similar structural characteristics, while different network attacks have significant structural differences. The purpose of highlighting the structural characteristics of network traffic samples is to determine the presence of privacy- and category-sensitive features.

For packet samples, their file format complies with the network transmission protocol, providing structural characteristics to the features and making it possible to identify sensitive features by the self-attention mechanism.

For session samples, which store features in a tabular format, sensitive features can be identified based on feature meanings. Of course, sensitive features of session samples can also be identified by the self-attention mechanism.

B. FDSM

FDSM achieves the following.

- 1) *Differential privacy* of the training dataset $\mathcal{S}_{\text{train}}$ should be guaranteed before the training samples are input into the model.
- 2) *Adversarial robustness* of the testing dataset $\mathcal{S}_{\text{test}}$ should be guaranteed before the testing samples are input into the model.

The definition of FDSM is as follows.

Definition 1 (FDSM): A randomized mechanism $M : \mathcal{X} \rightarrow \tilde{\mathcal{X}}$ satisfies $(\alpha, \gamma, \beta, \epsilon)$ -FDSM if for any similar sample feature vectors $\mathbf{x}_i, \mathbf{x}'_i \in \mathcal{X}$

$$D_\beta(f(M(\mathbf{x}_i)) || f(M(\mathbf{x}'_i))) \leq \epsilon, \text{ s.t. } d_\alpha(\mathbf{x}_i, \mathbf{x}'_i) \leq \gamma \quad (1)$$

where ϵ is the safety budget under β -divergence. As ϵ decreases or β increases and γ increases or α decreases, the machine learning model is safer. $\mathbf{x}'_i$ is any sample feature vector whose difference with $\mathbf{x}_i$ satisfies $d_\alpha(\mathbf{x}_i, \mathbf{x}'_i) \leq \gamma$. γ is the maximum difference between similar samples $\mathbf{x}_i$ and $\mathbf{x}'_i$ under the α -metric. $\mathcal{X}$ is all possible values of the sample feature vectors $\mathbf{x}_i$. M is a randomization mechanism for the feature vector $\mathbf{x}_i$. f is the feature distribution of $\mathcal{X}$ obtained by the machine learning model F .

FDSM is derived from differential privacy and adversarial robustness, and the derivation process is described in Appendix A (see the supplementary material).

IV. HIERARCHICALDP

In this section, we propose a feature hierarchization strategy for samples, analyze the correlation between different levels of features with differential privacy or adversarial robustness, and induce the theoretical basis of HierarchicalDP. Then, we introduce the attention mechanism network for computing feature weights and explain the implementation of HierarchicalDP using Laplace noise as an example.

A. Feature Hierarchization Strategy

Existing differential privacy mechanisms in machine learning, such as Gaussian mechanisms, face the curse of dimensionality. The noise in high-dimensional settings is unbearably considerable and severely affects model performance [1]. An intuitive question is whether differential privacy can be achieved by adding noise to only a subset of sample features.

Due to the structured characteristics of network traffic samples, whether packets or sessions, the assumption of sparsity in critical features can be fully exploited. Only a subset of sample features is relevant to differential privacy or adversarial robustness. In contrast, most features are nearly irrelevant to privacy or robustness. For instance, the protocol feature is always TCP in the semiconnect scanning traffic category. Obfuscating this feature does not contribute to privacy protection. Therefore, the primary objective is to identify the key features of differential privacy and adversarial robustness. To achieve this, we propose a feature hierarchization strategy. The following four levels are sufficient for a nonredundant partitioning of sample features, while more detailed levels would result in unnecessary computational overhead.

Feature hierarchization strategy. For any d -dimensional sample $\mathbf{x}_i \sim Y_c$, there exists four levels of features.

- 1) *Level-1 features* (intra-class critical features) are strongly correlated to the Y_c category, with dimension number $j \in S^1 \sim Y_c$.
- 2) *Level-2 features* (interclass critical features) are weakly correlated to the Y_c category but strongly correlated to any other category, with dimension number $j \in S^2 \sim Y_c$.
- 3) *Level-3 features* (intra-class noncritical features) are weakly correlated to any category but strongly correlated to a particular sample $\mathbf{x}_i$, with dimension number $j \in S^3 \sim Y_c$.
- 4) *Level-4 features* (interclass noncritical features) are weakly correlated to any category or sample, with dimension number $j \in S^4 \sim Y_c$.

Level-1 features are decisive for sample $\mathbf{x}_i$ to belong to category Y_c , so their values are generally stable. Level-2 features are related to adversarial robustness because adjusting these features causes the machine learning model to infer $\mathbf{x}_i$ to other categories. Level-3 features are related to differential privacy because these features carry information unique to $\mathbf{x}_i$. Level-4 features take almost the same value in any sample. Accordingly, we define several foundational concepts in HierarchicalDP.

Definition 2 (Range and sensitivity of HierarchicalDP): Let there be n samples in any dataset S , each sample with d -dimensional features. There are m categories in S , and

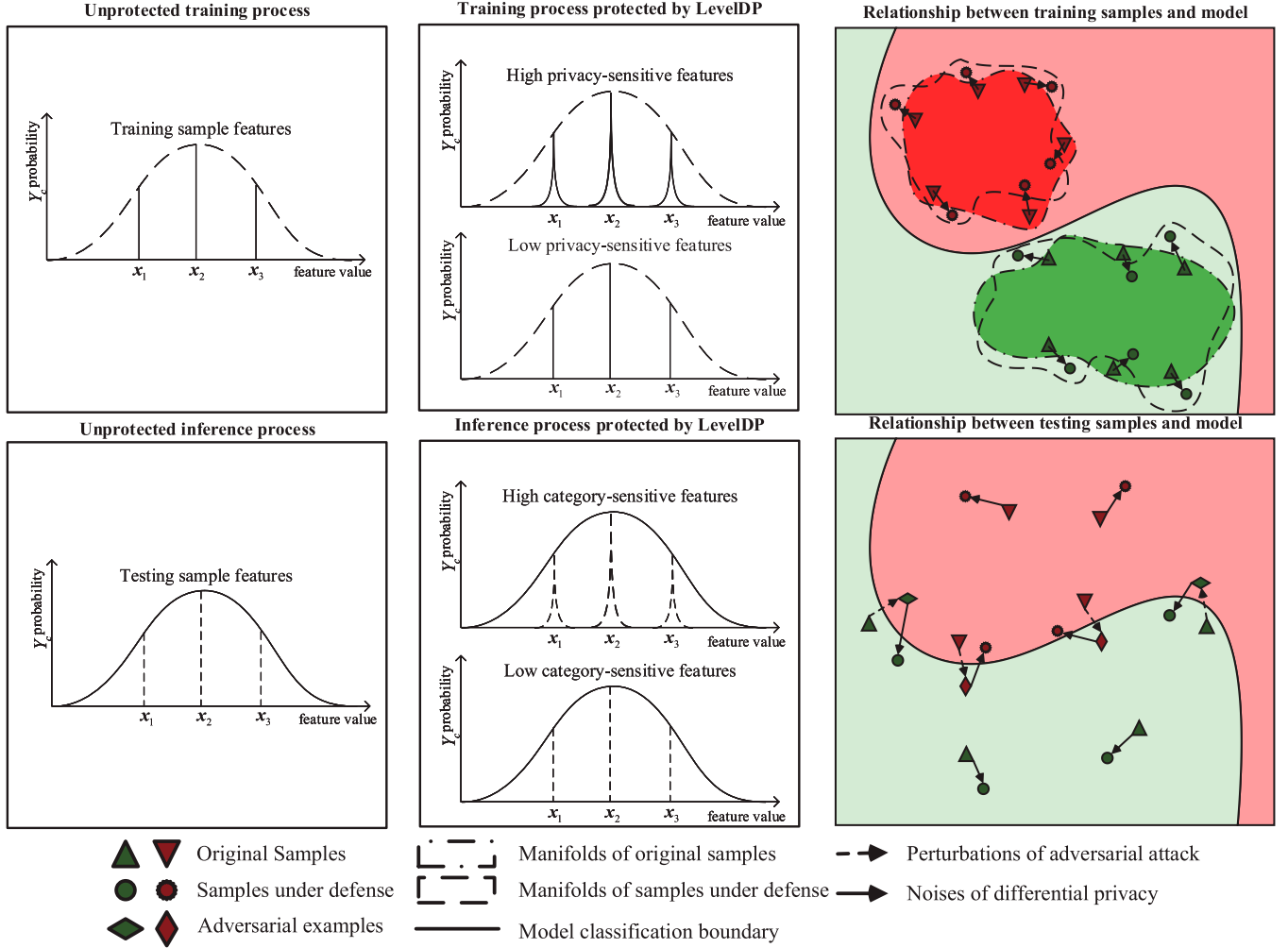


Fig. 2. Schematic illustration of the defense effects of HierarchicalDP on the model training and inference process. During model training, high privacy-sensitive features of training samples are protected to defend against membership inference. During model inference, high category-sensitive features of testing samples are protected to defend against adversarial attack.

each category Y_c has n_c samples, $n = \sum_{c=1}^m n_c$. For any sample, $\mathbf{x}_i (1 \leq i \leq n)$ has (for training sample) or implicitly has (for testing sample) a category label $Y_c (1 \leq c \leq m)$. Each category Y_c possesses a feature weight vector $\mathbf{w} \sim Y_c = (w_1, w_2, \dots, w_d)$, $\sum_{j=1}^d w_j = 1$. The weight vector of the universal sample set $\mathcal{X}$ is $\bar{\mathbf{w}} = \sum_{c=1}^m \frac{n_c}{n} \mathbf{w} \sim Y_c$. f denotes the information of the feature distribution obtained by the machine learning model F from $\mathbf{x}_i \sim Y_c$.

- 1) The global sample range in $\mathcal{S}$ is $R = \max_i d_\alpha(\mathbf{x}_i, \mathbf{x}'_i)$.
- 2) For the class Y_c , the intraclass sample range in $\mathcal{S}$ is $R \sim Y_c = \max_i d_\alpha(\mathbf{x}_i, \mathbf{x}'_i)$, where $\mathbf{x}_i, \mathbf{x}'_i \sim Y_c$.
- 3) The global sample sensitivity in $\mathcal{S}$ is $\Delta \bar{f} = \max_i d_\alpha(\bar{\mathbf{w}} \odot \mathbf{x}_i, \bar{\mathbf{w}} \odot \mathbf{x}'_i)$.
- 4) For the class Y_c , the intraclass sample sensitivity in $\mathcal{S}$ is $\Delta f \sim Y_c = \max_i d_\alpha(\mathbf{w} \odot \mathbf{x}_i, \mathbf{w} \odot \mathbf{x}'_i)$, where $\mathbf{x}_i, \mathbf{x}'_i, \mathbf{w} \sim Y_c$.
- 5) For the j th feature of class $Y_c (1 \leq j \leq d)$, the intraclass feature sensitivity in $\mathcal{S}$ is $\Delta f_j \sim Y_c = \max_i d_\alpha(w_j x_{ij}, w_j x'_{ij})$, where $\mathbf{x}_i, \mathbf{x}'_i \sim Y_c$ and w_j is the j th dimension of $\mathbf{w} \sim Y_c$.

- 6) For the l -th of class $Y_c (1 \leq l \leq 4)$, the intraclass L - l sensitivity in $\mathcal{S}$ is $\Delta f^l \sim Y_c = \max_{i,j} d_\alpha(w_j x_{ij}, w_j x'_{ij})$, where $\mathbf{x}_i, \mathbf{x}'_i \sim Y_c$, $j \in S^l \sim Y_c$ and w_j is the j th dimension of $\mathbf{w} \sim Y_c$.

Where w_j denotes the importance of the j th feature for $\mathbf{x}_i$ to belong to class Y_c . Considering the samples in $\mathcal{S}$ as random vectors, the sample $\mathbf{x}_i \sim Y_c$ has d -dimensional features $x_{ij} (1 \leq j \leq d)$, where the j th feature x_{ij} takes a value that follows the probability distribution $p_j \sim Y_c$, then the sample $\mathbf{x}_i \sim Y_c$ follows the joint probability distribution $p_{1,2,\dots,d} \sim Y_c$.

HierarchicalDP protects *data privacy* by ensuring that samples of the same class are indistinguishable, then $\gamma = \Delta f \sim Y_c$ in $(\alpha-\gamma, \beta-\epsilon)$ -FDSM. It assumes that the number of samples in each category $\min_c n_c \gg 1$ and $\Delta f \sim Y_c \ll \Delta \bar{f}$. Intraclass privacy protection can reduce the noise variance with a fixed privacy budget ϵ . As shown in Fig. 2, during the model training process, HierarchicalDP focuses on protecting privacy-sensitive features of training samples. Since different samples of the same class have significant differences in level-2 and level-3 features, the privacy correlation of these features is high.

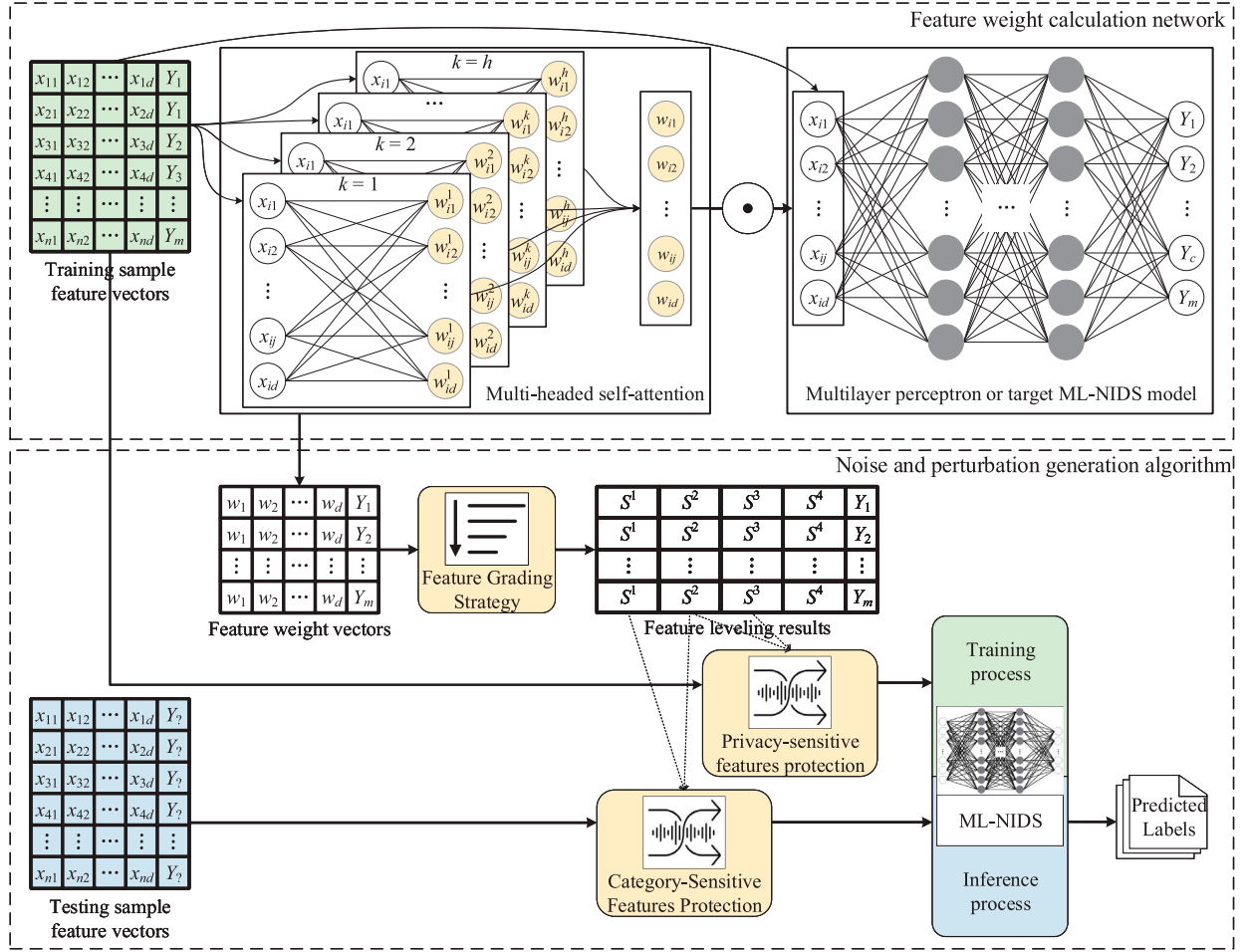


Fig. 3. Architecture of HierarchicalDP. It is divided into the feature weight calculation network and the noise and perturbation generation algorithm. The former calculates the importance weights of each feature for different categories. The latter levels the features according to the weights and adds different noises to the training and testing samples based on the leveling results to achieve differential privacy and adversarial robustness.

HierarchicalDP enhances *inference robustness* by ensuring that samples do not change their feature distributions due to minor perturbations, then $\gamma = \Delta f \sim Y_c$ in $(\alpha\text{-}\gamma, \beta\text{-}\epsilon)$ -FDSM. It assumes that the sample feature distribution of the same category in S_{test} is centralized and $R \sim Y_c \ll R$. Adding intraclass perturbations to samples can destroy the structure of the adversarial perturbation without changing the feature distribution too much. As shown in Fig. 2, during the model inference process, HierarchicalDP focuses on protecting category-sensitive features of testing samples. Changes in level-1 and level-2 features are more likely to cause class changes, hence the higher class correlation of these features.

B. HierarchicalDP

The architecture of the HierarchicalDP is shown in Fig. 3, which is divided into two parts: the feature weight calculation network and the noise and perturbation generation algorithm.

1) *Feature Weight Calculation Network*: Feature weight calculation network is based on the method proposed by Škrlj et al. [25]. The difference is that our method can compute feature weight vectors for different categories. The feature weight

calculation network is divided into the MSA and the multilayer perceptron (MLP). The MLP can be replaced with the target machine learning model to compute more accurate feature weights. The MSA can be represented as

$$\text{MSA}(\mathbf{x}_i) = \frac{1}{h} \sum_{k=1}^h \text{softmax}(\mathbf{W}^k \mathbf{x}_i + \mathbf{b}^k)$$

$$\text{softmax}(\mathbf{x}_i) = \frac{e^{x_{ij}}}{\sum_{j=1}^d e^{x_{ij}}}$$

where $\mathbf{x}_i = (x_{i1}, x_{i1}, \dots, x_{id})$ is the d -dimensional feature vector of the i th input sample. $\mathbf{x}_i$ is multiplied by the matrix $\mathbf{W}^k$ and adds the bias vector $\mathbf{b}^k$, essentially a spatial mapping of $\mathbf{x}_i$. The mapping result is input to the softmax function, which changes each vector dimension into a weight in the interval $[0, 1]$. This process is repeated h times to obtain h weight vectors $\mathbf{w}_i^1, \mathbf{w}_i^2, \dots, \mathbf{w}_i^h$. The final weight vector $\mathbf{w}_i$ of $\mathbf{x}_i$ is the average of the h weight vectors.

The MSA is a parametric Nadaraya–Watson kernel regression using a radial basis function. The feature vector $\mathbf{x}_i$ is mapped to a new space using matrix $\mathbf{W}^k$ and vector $\mathbf{b}^k$. The softmax function weighted the average of the mapped vectors

to obtain the feature weight vectors. The weights are calculated multiple times and averaged to mitigate the effect of the initialization of $\mathbf{W}^k$ and $\mathbf{b}^k$ on the results.

The MSA is connected to the MLP using the $\odot$ (Hadamard product). The weight vectors $\mathbf{w}_i$ are multiplied by the corresponding dimensions of the feature vectors $\mathbf{x}_i$. The weighted feature vectors are input into the MLP. The parameters of MSA ($\mathbf{W}^k$ and $\mathbf{b}^k$) and MLP are trained to optimize the cross-entropy loss to make the feature weights more accurate.

When the network training is completed, the MLP can be removed. Samples in $\mathcal{S}_{\text{train}}$ are reinput into the MSA. Each $\mathbf{x}_i \in \mathcal{S}_{\text{train}}$ outputs a d -dimensional weight vector $\mathbf{w}_i$. Let there be n samples in $\mathcal{S}_{\text{train}}$ and n_c samples for each category Y_c , $n = \sum_{c=1}^m n_c$. The weight vector $\mathbf{w} \sim Y_c$ of the Y_c category and the weight vector $\bar{\mathbf{w}}$ of the universal sample set $\mathcal{X}$ are as follows:

$$\mathbf{w} \sim Y_c = \frac{1}{n_c} \sum_{\mathbf{x}_i \sim Y_c} \text{MSA}(\mathbf{x}_i)$$

$$\bar{\mathbf{w}} = \frac{1}{n} \sum_{i=1}^n \text{MSA}(\mathbf{x}_i).$$

2) Noise and Perturbation Generation Algorithm:

The noise and perturbation generation algorithm contains three parts: the feature hierarchization strategy algorithm, the privacy-sensitive features protection algorithm, and category-sensitive features protection algorithm.

The feature weight calculation network can provide the weight of each feature for each category. The security budget ϵ and feature sensitivity Δf_j determine how the feature levels are classified based on the weights w_j . The noise added to each level feature can be adjusted according to the problem. This article focuses on the most common Laplace noise [13], but the methods involved can be extended to other types of noise.

The differential privacy can result to different probabilities for the two queries generating the same value smaller than e^ϵ . The different probabilities are resulted from adding the random noise to the output. The reason of HierarchicalDP adding noise to only a subset of features to achieve $(\alpha-\gamma, \beta-\epsilon)$ -FDSM is as follows.

A machine learning model F gets a randomly returned feature vector $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id})$ for a query of $\mathcal{S}_{\text{train}}$ or $\mathcal{S}_{\text{test}}$. Suppose the difference in the probability of the two features x_{ij}, x'_{ij} returning the same value for two queries is initially smaller than e^ϵ ; adding noise to these features is not needed. So ϵ and Δf_j determine which x_{ij} does not require adding noise. In particular, if the $\epsilon \rightarrow \infty$ or $\Delta f_j \rightarrow 0$, any x_{ij} does not need to add noise. Algorithm 1 shows how to hierarchize the features of network traffic samples based on their feature weights. Algorithm 2 shows how HierarchicalDP protects the privacy-sensitive features of network traffic samples. Algorithm 3 shows how HierarchicalDP protects the category-sensitive features of network traffic samples.

The proof that Algorithms 2 and Algorithm 3 satisfy $(\alpha-\gamma, \beta-\epsilon)$ -FDSM is shown in Appendix C (available online). The HierarchicalDP mechanism does not add noise to features with high weights, as the features with lower weights often

Algorithm 1 Feature Hierarchization Strategy

Input: weight vectors $\mathbf{w} \sim Y_c$, safety budget ϵ , intra-class feature sensitivities $\Delta f_j \sim Y_c$

Output: feature hierarchization results $\mathcal{S} \sim Y_c = \{S^1, S^2, S^3, S^4\}$

```

1: for  $c = 1$  to  $m$  do
2:   Initialize  $S_{\text{all}} = \{1, 2, \dots, d\}$ ;
3:   for  $j = 1$  to  $d$  do
4:     if  $w_j \geq \epsilon$  then  $\triangleright$  the  $j$ -dimension of  $\mathbf{w} \sim Y_c$ 
5:       put  $j$  into  $S^1$ ;
6:       put  $j$  into  $S^2_{\text{temp}}$ ;
7:     else if  $\Delta f_j \leq \frac{\epsilon}{d}$  then  $\triangleright \Delta f_j \sim Y_c$ 
8:       put  $j$  into  $S^4$ ;
9:     end if
10:  end for
11: end for
12: for  $c = 1$  to  $m$  do
13:    $S^2 \leftarrow S^2_{\text{temp}} - S^1$ ;
14:    $S^3 \leftarrow S_{\text{all}} - S^1 - S^2 - S^4$ ;
15: end for
16: return  $\mathcal{S} \sim Y_c$ ;
```

Algorithm 2 Privacy-Sensitive Features Protection

Input: training set $\mathcal{S}_{\text{train}}$, privacy budget ϵ , intra-class L_{v3} feature sensitivity $\Delta f_j^{2 \vee 3} \sim Y_c$, feature hierarchization results $\mathcal{S} \sim Y_c$

Output: training set $\tilde{\mathcal{S}}_{\text{train}}$ satisfying $(\alpha-\gamma, \beta-\epsilon)$ -FDSM

```

1: for  $c = 1$  to  $m$  do
2:   for  $i = 1$  to  $n_c$  do
3:     for  $j = 1$  to  $d$  do
4:       if  $j \in S^2$  or  $j \in S^3$  then
5:         noise  $z \sim \text{Laplace}(0, \frac{\Delta f_j^{2 \vee 3}}{\epsilon})$ ;
6:          $\tilde{x}_{ij} \leftarrow x_{ij} + z$ ;  $\triangleright \mathbf{x}_i \in \mathcal{S}_{\text{train}}, \tilde{\mathbf{x}}_i \in \tilde{\mathcal{S}}_{\text{train}}$ 
7:       else
8:          $\tilde{x}_{ij} \leftarrow x_{ij}$ ;
9:       end if
10:    end for
11:  end for
12: end for
13: return  $\tilde{\mathcal{S}}_{\text{train}}$ ;
```

lead to privacy disclosure. It may be counterintuitive. However, it is easy to understand when considering that overfitting is one of the fundamental causes of membership inference and adversarial attack. Overfitting is precisely caused by models learning too much detail about irrelevant features. On the one hand, the adversarial robustness of models trained with $\tilde{\mathcal{S}}_{\text{train}}$ is also improved. Algorithm 2 can be used jointly with Algorithm 3 to achieve a more robust model security protection. On the other hand, depending on the actual requirements, using only one of Algorithm 2 or Algorithm 3 to achieve more lightweight security protection.

Algorithm 3 Category-Sensitive Features Protection

Input: training set $\mathcal{S}_{\text{test}}$, robustness budget ϵ , intra-class L_{v2} feature sensitivity $\Delta f_j^{1 \vee 2} \sim Y_c$, feature hierarchization results $\mathcal{S} \sim Y_c$

Output: testing set $\tilde{\mathcal{S}}_{\text{test}}$ satisfying $(\alpha\text{-}\gamma, \beta\text{-}\epsilon)\text{-FDSM}$

```

1: for  $i = 1$  to  $n$  do
2:   for  $j = 1$  to  $d$  do
3:     if  $j \in \forall S^1 \sim Y_c$  then
4:       perturbation  $z \sim \text{Laplace}(0, \frac{\max_c \Delta f^{1 \vee 2}}{\epsilon})$ ;
5:        $\tilde{x}_{ij} \leftarrow x_{ij} + z$ ;  $\triangleright \mathbf{x}_i \in \mathcal{S}_{\text{test}}, \tilde{\mathbf{x}}_i \in \tilde{\mathcal{S}}_{\text{test}}$ 
6:     else
7:        $\tilde{x}_{ij} \leftarrow x_{ij}$ ;
8:     end if
9:   end for
10: end for
11: return  $\tilde{\mathcal{S}}_{\text{test}}$ ;

```

V. EXPERIMENTS

We evaluate HierarchicalDP by answering four key questions.

- 1) *Q1*: Does HierarchicalDP correctly partition sample features into security levels?
- 2) *Q2*: Does HierarchicalDP achieve optimal defense effectiveness by adding noise to each security level feature?
- 3) *Q3*: How effective is HierarchicalDP in defending against membership inference and adversarial attacks? How does it compare with other best-effort defenses?
- 4) *Q4*: How does HierarchicalDP perform on datasets other than network traffic?

To address the above questions, we selected a packet sample dataset (USTC-TFC2016) and a session sample dataset (CICIDS2017) for experimentation. We evaluated the performance of both datasets using four typical machine learning models. We compared the defense effect of HierarchicalDP against two types of membership inference and two types of adversarial attacks. Section V-A describes the datasets, prior defenses, and our evaluation methodology. Subsequent sections address each question in turn. The evaluation code is available at <https://github.com/liuguangrui-hit/HierarchicalDP>.

Evaluation highlights: HierarchicalDP provides reasonable and quantifiable privacy budgets and robustness budgets for different models on network traffic datasets. This is the first work that unifies and implements differential privacy and adversarial robustness. HierarchicalDP provides the privacy budget $\epsilon \leq 0.2$ and the robustness budget $\epsilon \leq 0.2$ under D_∞ divergence, with the sensitivity $\gamma \leq 1$ under d_1 metric. Comparing HierarchicalDP to the best-effort defenses on membership inference, HierarchicalDP models give better privacy protection (e.g., 69.2% versus 50.0% MIR for CICIDS2017 on LSTM) and similar ACC fluctuations (less than 3.2%). Comparing HierarchicalDP to the best-effort defenses on adversarial attacks, HierarchicalDP models give better adversarial robustness (e.g., 18.7% versus 10.9% AER for MNIST on CNN) and similar ACC fluctuations (less than 4.2%).

A. Methodology

1) *Datasets*: We evaluated the defensive effectiveness of HierarchicalDP using two network traffic datasets listed in Table II.

USTC-TFC2016 [26] is a network traffic classification task containing 20 categories and 65k samples. Each sample is composed of $28 \times 28 = 784$ features and a classification label. We randomly selected ten categories for testing.

CICIDS2017 [27] is an intrusion detection task composed of malicious network traffic containing 22 categories and 238 000 samples. Each sample is composed of 78 features and a classification label. Due to the uneven distribution of samples in each category, we integrated the categories according to the method in [28].

2) *Models*: We evaluated the defensive effectiveness of HierarchicalDP using four machine learning models.

MLP is a three-layer neural network consisting of an input, hidden, and output layer. It is the simplest feedforward neural network structure.

Deep neural network (DNN) is a deeper feedforward neural network with one input layer, eight hidden layers, and one output layer.

Convolutional neural network (CNN) is a neural network suitable for image recognition, consisting of one input layer, five convolutional layers, five pooling layers, three fully connected layers, and one output layer. The size of the convolutional kernel is 3×3 .

LSTM is a recurrent neural network suitable for processing temporal tasks, and we used the network structure from [29].

We trained three baseline models for each dataset in our experiments. Table II shows the training and testing set for each dataset and the corresponding accuracy of each baseline model. The USTC-TFC2016 dataset structure resembles image data, making it suitable for CNN models. The CICIDS2017 dataset structure exhibits strong temporal characteristics, making it suitable for LSTM models.

3) *Evaluation Metrics*: We use three metrics to evaluate the defense effectiveness of HierarchicalDP:

MIR represents the probability of inferring whether a given sample is from the training set through membership inference.

AER represents the probability that an adversarial example can escape the recognition of the model.

ACC represents the classification accuracy of the model on the testing set.

4) *Attack Methodology*: We randomly selected 1000 samples for each dataset as cases for membership inference and adversarial attacks, among which 500 samples were from the testing set and 500 samples were from the training set.

For membership inference, we choose two methods based on the loss function of the target model and the predictions of the shadow model. Modification of prediction entropy (MPE) [30] is a prediction-based inference attack based on prediction entropy modification. Enhanced membership inference (EMI) [31] is a shadow model-based EMI attack.

For adversarial attacks, we choose two methods with higher transferability and richer attack forms. Nesterov iterative fast

TABLE II
EVALUATION DATASETS AND BASELINE MODELS

Symbol	Feature Size	Training Set Size	Testing Set Size	Target Labels	Model Architecture (Baseline Accuracy)
USTC-TFC2016	784	6k	3k	10	MLP (0.859), DNN (0.897), CNN (0.916)
CICIDS2017	78	20k	10k	7	MLP (0.854), DNN (0.887), LSTM (0.901)

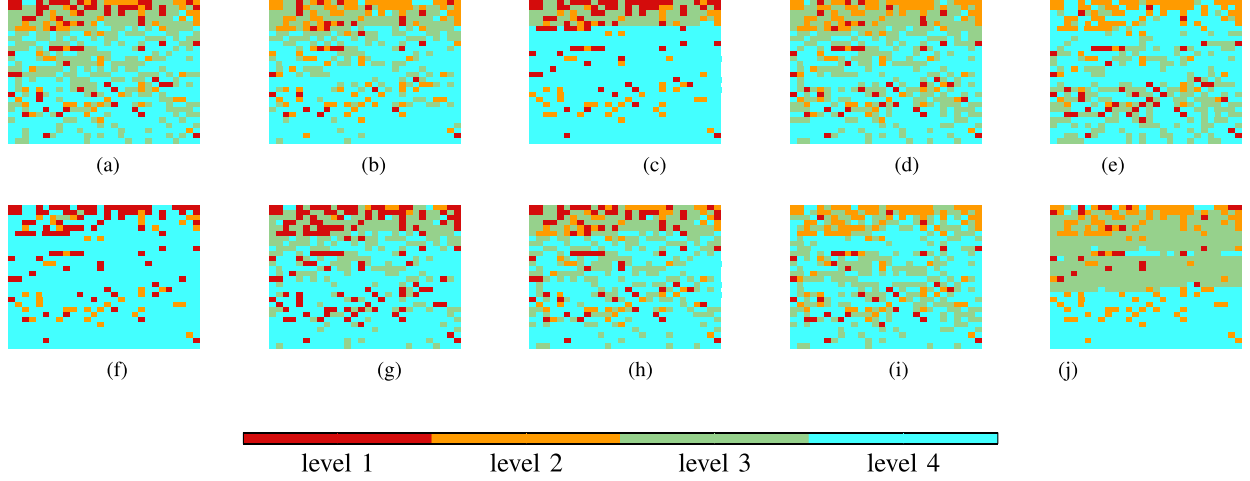


Fig. 4. Grading results of different class features in USTC-TFC2016: (a) BitTorrent; (b) Facetime; (c) FTP; (d) Gmail; (e) Miuref; (f) MySQL; (g) Neris; (h) Nsis-ay; (i) Outlook; and (j) WorldOfWarcraft.

gradient sign method (NIFGSM) [32] is an iterative adversarial attack based on the Nesterov accelerated gradient method with high transferability of adversarial examples. AutoAttack project gradient descent (APGD) [33] is an extended version of the PGD attack that integrates multiple attack methods.

5) *Prior Defenses for Comparison:* We compared HierarchicalDP, a defense method based on Laplace noise, with previous defense methods.

For membership inference defense, we choose two methods involving loss function optimization and model output correction. Adversarial regularization (AdvReg) [34] is an enhanced adversarial training technique that minimizes the prediction loss of the model and maximizes the cost of inference attacks. *MemGuard* [35] is a defense technique that adds noise to the confidence to create an adversarial deception against membership inference attackers.

For adversarial attacks defense, we choose two methods involving sample feature smoothing and reverse adversarial perturbations. Denoised smoothing (DenSmo) [36] is a technique that trains a denoiser for the target model and smooths sample features to eliminate adversarial perturbations. Antiadversaries (AntiAdv) [37] is a technique that adds a noise vector with the opposite direction of adversarial perturbations to testing samples to counteract their adversarial effects.

B. Security Level Partitioning Results (Q1)

Q1: Does HierarchicalDP correctly partition sample features into security levels?

We used the feature hierarchization strategy to partition the features of each dataset. First, we normalized the sample feature vectors. Then, we tried to make the feature differences as

obvious as possible and the number of features balanced while ensuring the privacy and robustness budget $\epsilon \leq 0.2$. Figs. 4 and 5 show the feature hierarchization results for each dataset.

For USTC-TFC2016 ($\epsilon = 0.2$), as shown in Fig. 4, the level-1 and level-2 features are concentrated in the upper part of the image. In contrast, the level-3 and level-4 features are in the lower part. This is because the USTC dataset converts the PCAP files into 28×28 image pixels in hexadecimal form and append 0x00 if the PCAP file is shorter than 784 bytes. The feature hierarchization result graph reflects this process well. The level-1 features account for 7.8%, the level-2 features account for 9.4%, the level-3 features account for 26.6%, and the level-4 features account for 56.1%.

For CICIDS2017 ($\epsilon = 0.2$), as shown in Fig. 5, since it is not an image sample, its features do not have a positional relationship. Due to the brief nature of CICIDS features, the sample dataset tends to focus on distinguishing between malicious and benign instances for better intrusion detection. Consequently, the differences among malicious samples are relatively small, but the distinction between malicious and benign samples is significant. However, attack traffic with similar functions (such as Dos and DDoS) also exhibits more similar feature distributions. The level-1 features account for 19.4%, the level-2 features account for 26.7%, the level-3 features account for 16.5%, and the level-4 features account for 37.4%.

C. Impact of Noise (Q2)

Q2: Does HierarchicalDP achieve optimal defense effectiveness by adding noise to each security level feature?

We choose the USTC-TFC2016 dataset and the MLP model as an example.

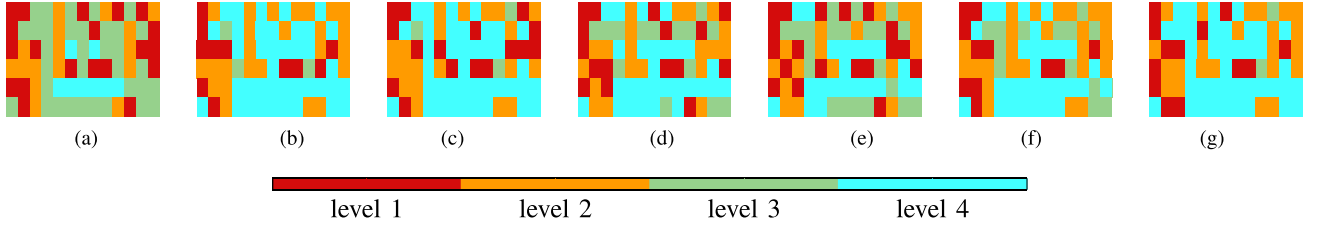


Fig. 5. Grading results of different class features in CICIDS2017: (a) Benign; (b) Bot; (c) Bruteforce; (d) DDoS; (e) Dos; (f) PortScan; and (g) web-attack.

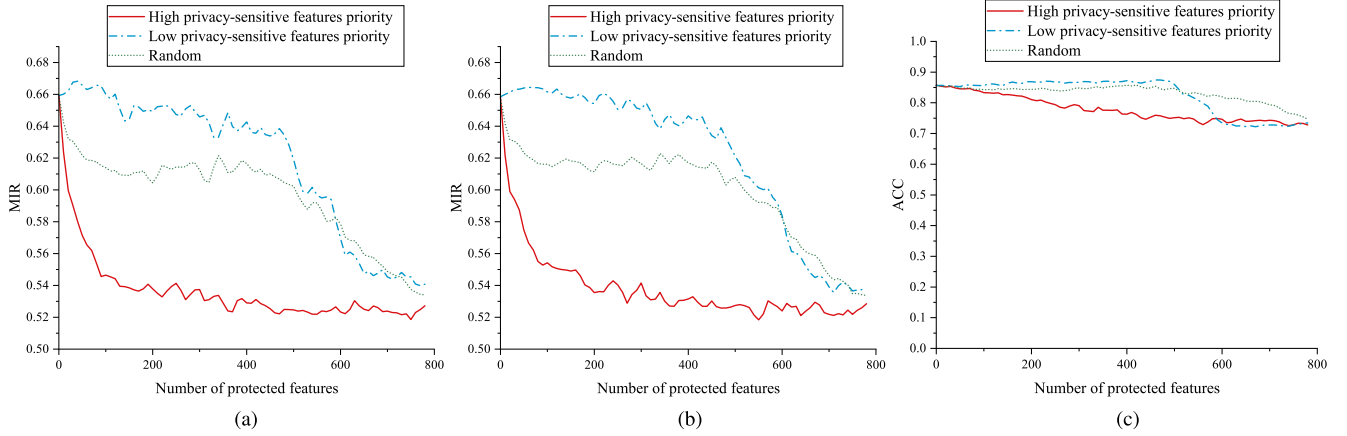


Fig. 6. Trends of MIR and ACC as the number of protected features in training samples increases (USTC-TFC2016 on MLP). The number of features is increased in three ways, increasing ten each time: (a) MIR of MPE; (b) MIR of EMI; and (c) ACC of MLP.

1) *Validation of Feature Privacy Relevance*: During model training, HierarchicalDP noise is gradually added to more features in the training samples input into the model in the following three orders, and the changes in ACC and MIR are observed.

- a) *High privacy-sensitive features priority*: HierarchicalDP noise is added to the second, third, first, and fourth level features of the training samples in order, with features arranged in descending order of weight and increasing the number of protected features by ten each time.
- b) *Low privacy-sensitive features priority*: HierarchicalDP noise is added to the fourth, first, third, and second level features of the training samples in order, with features arranged in ascending order of weight and increasing the number of protected features by ten each time.
- c) *Random*: Add HierarchicalDP noise to features randomly, increasing the number of protected features by ten each time.

2) *Validation of Feature Category Relevance*: During model inference, HierarchicalDP noise is gradually added to more features in the testing samples input into the model in the following three orders, and the changes in the ACC and AER are observed.

- a) *High category-sensitive features priority*: HierarchicalDP noise is added to the first, second, third, and fourth level features of the testing samples in order, with features

arranged in descending order of weight and increasing the number of protected features by one each time.

- b) *Low category-sensitive features priority*: HierarchicalDP noise is added to the fourth, third, second, and first level features of the testing samples in order, with features arranged in ascending order of weight and one protected feature added each time.
- c) *Random*: Add HierarchicalDP noise to features randomly, increasing the number of protected features by one each time.

As shown in Fig. 6, MIR decreases the fastest when protecting high privacy-sensitive priority and the slowest when protecting low privacy-sensitive priority. When protecting high privacy-sensitive priority, after protecting all level-2 and level-3 features (the top 30%), adding more protected features leads to MIR decrease slowly. However, adding HierarchicalDP noise to the training samples significantly impacts the ACC. Therefore, only HierarchicalDP noise to level-2 and level-3 features can protect privacy without severely affecting the ACC. When protecting low privacy-sensitive features priority, MIR decreases rapidly after protecting the bottom 30% of the features. This is mainly due to two reasons: 1) HierarchicalDP starts to protect high privacy-sensitive features (level-3) and 2) enough features have been protected. When protecting low privacy-sensitive features, ACC begins to rapidly decrease after protecting the

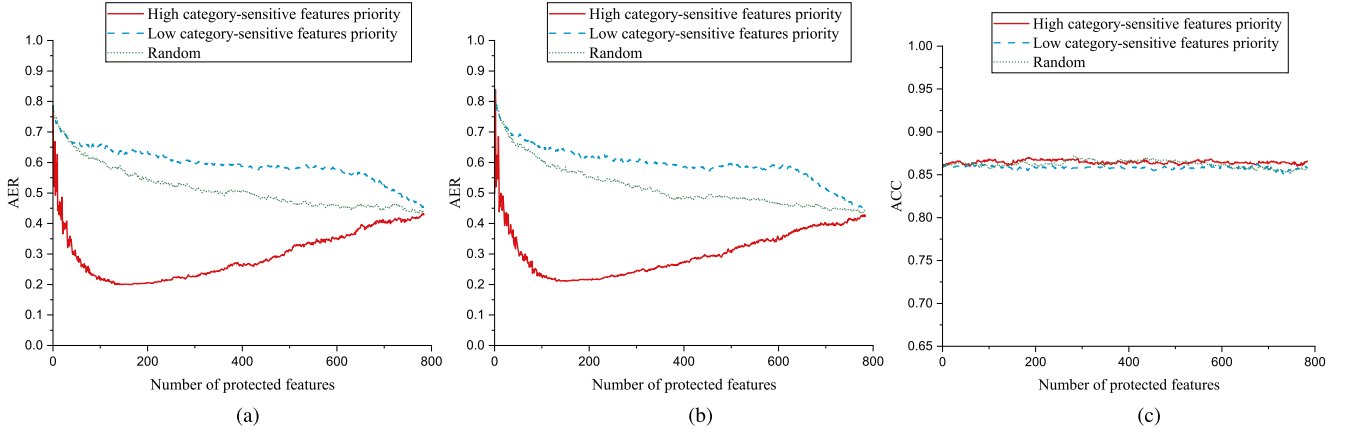


Fig. 7. Trends of AER and ACC as the number of protected features in testing samples increases (USTC-TFC2016 on MLP). The number of features is increased in three ways, increasing one each time: (a) AER of NIFGSM; (b) AER of APGD; and (c) ACC of MLP.

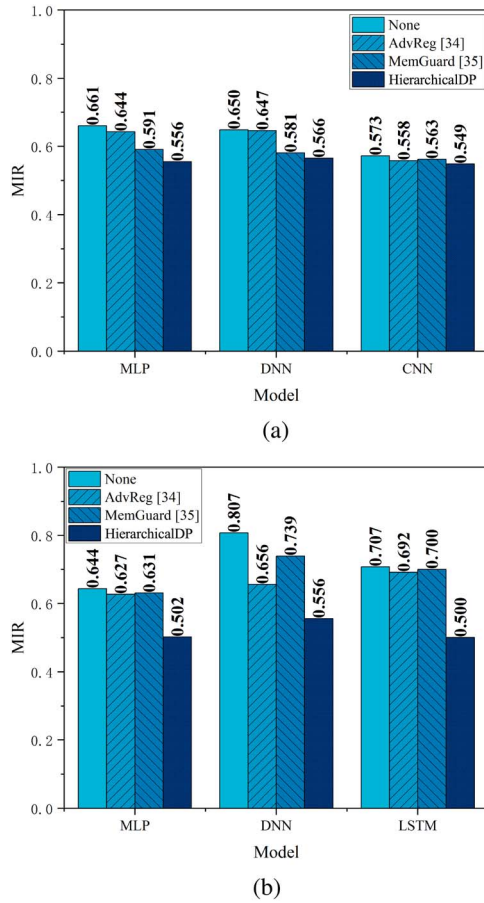


Fig. 8. Comparison of MIR for different models under various member inference defenses against MPE [30]: (a) USTC and (b) CICIDS.

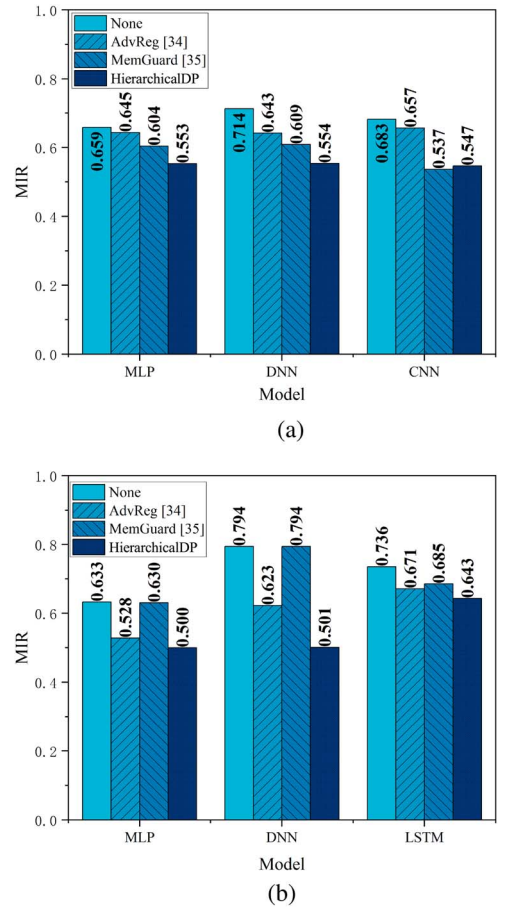


Fig. 9. Comparison of MIR for different models under various member inference defenses against EMI [31]: (a) USTC and (b) CICIDS.

bottom 40% of the features, mainly due to adding noise to high category-sensitive features (level-1).

As shown in Fig. 7, AER decreases the fastest when protecting high category-sensitive features priority and the slowest when protecting low category-sensitive features priority. Adding HierarchicalDP noise to the testing samples hardly

affects the ACC. When protecting high category-sensitive features priority, after protecting all level-1 features (the top 10%), increasing the number of protected features leads to a higher AER. This is because the adversarial samples are located on the model classification boundary, and adding noise to too many features can easily change the sample category.

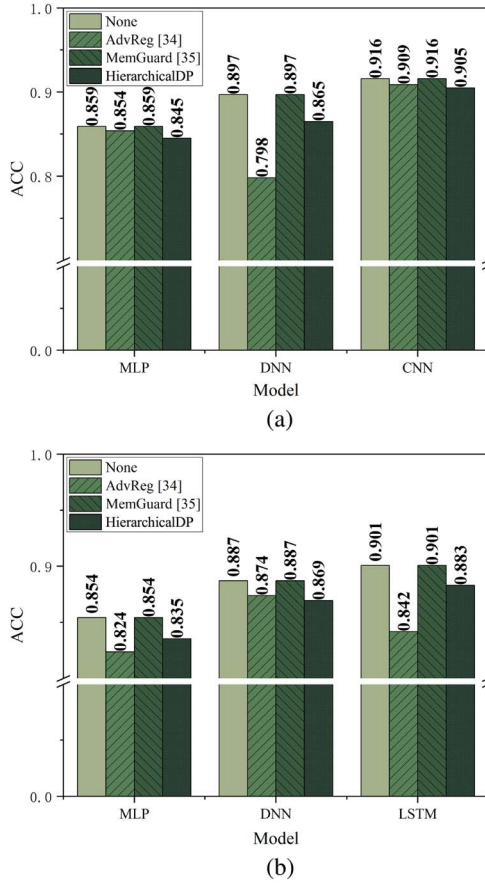


Fig. 10. Comparison of the impact of various member inference defenses on ACC: (a) USTC and (b) CICIDS.

Therefore, adding HierarchicalDP noise only to level-1 features can achieve the best defense effect against adversarial attacks.

D. Effectiveness of Defense (Q3)

Q3: How effective is HierarchicalDP in defending against membership inference and adversarial attacks? How does it compare with other best-effort defenses?

We applied HierarchicalDP protection to model input samples during training and inference. We test the differences between the model with and without defense in ACC, MIR, and AER. We also compared the defense effectiveness of HierarchicalDP with other membership inference defenses and adversarial attack defenses.

Regarding the privacy of training samples, Fig. 8 compares the effectiveness of various defense methods against MPE attacks. HierarchicalDP shows slightly improved performance on the USTC-TFC2016 dataset compared to existing methods, reducing MIR by an average of 4.0% on various models. HierarchicalDP offers significantly improved performance on the CICIDS2017 dataset, reducing MIR by an average of 15.5% on various models. This is because the CICIDS2017 dataset has a stronger structural regularity. A more accurate calculation of feature weights is caused by more explicit features.

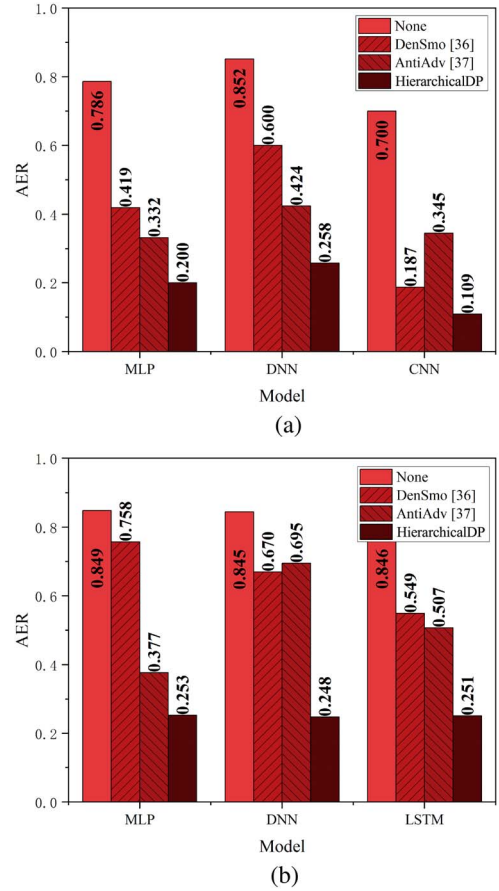


Fig. 11. Comparison of AER for different models under various adversarial attack defenses against NIFGSM [32]: (a) USTC and (b) CICIDS.

Fig. 9 compares the defense effectiveness of various defense methods against EMI attacks. HierarchicalDP shows improved performance on the USTC-TFC2016 and CICIDS2017 datasets compared to existing methods, reducing MIR by an average of 8.6% on various models. Fig. 10 shows the impact of various defense methods on ACC. The effect of adding HierarchicalDP noise during model training on accuracy is similar to that of other methods, with fluctuations within 3.2%.

Regarding the robustness of testing samples, Fig. 11 compares the effectiveness of various defense methods against NIFGSM attacks. HierarchicalDP offers significantly improved performance on the USTC-TFC2016 and CICIDS2017 datasets, reducing AER by an average of 26.9% on various models. Fig. 12 compares the defense effectiveness of various defense methods against APGD attacks. HierarchicalDP shows slightly improved performance on the USTC-TFC2016 dataset compared to existing methods, reducing AER by an average of 10.8% on various models. HierarchicalDP offers significantly improved performance on the CICIDS2017 dataset, reducing AER by an average of 28.8% on various models. Fig. 13 shows the impact of various defense methods on ACC. The effect of adding HierarchicalDP noise during model training on accuracy is smaller than other methods, with fluctuations within 4.2%.

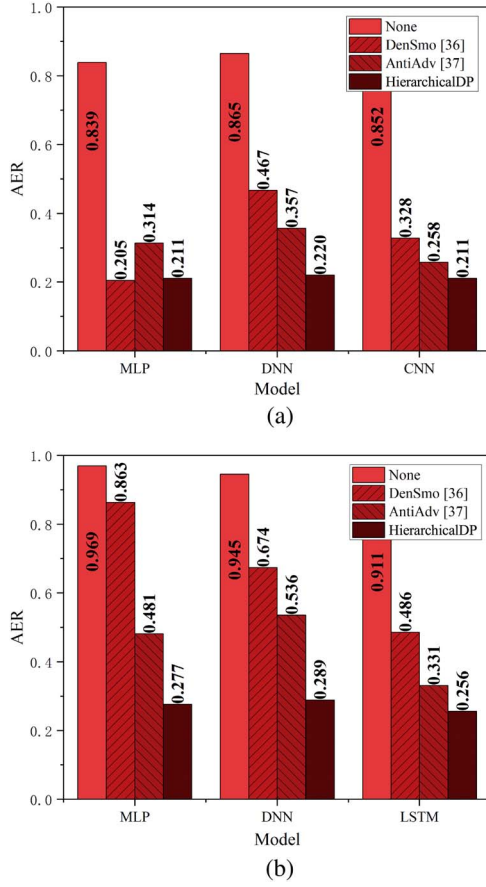


Fig. 12. Comparison of AER for different models under various adversarial attack defenses against APGD [33]: (a) USTC and (b) CICIDS.

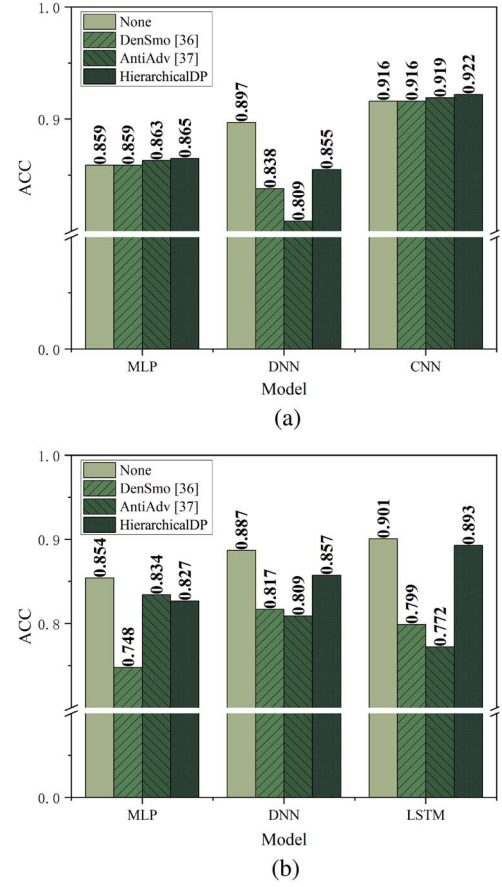


Fig. 13. Comparison of the impact of various adversarial attack defenses on ACC: (a) USTC and (b) CICIDS.

E. Extending Experiments (Q4)

Q4: How does HierarchicalDP perform on datasets other than network traffic?

Structured characteristics are not only present in network traffic samples but also in center-aligned image samples. To demonstrate this point, we conducted defense effectiveness tests of HierarchicalDP on both a center-aligned image dataset (MNIST) and a noncenter-aligned image dataset (CIFAR-10). The experiments utilized the MLP, DNN, and CNN models mentioned in Section V-A. Table III presents the training set and testing set configuration for each dataset, along with the corresponding accuracy of each baseline model. This section focuses on the defense effectiveness of HierarchicalDP across different image classifiers. The specific privacy budget settings and feature hierarchization results are shown in Appendix B (available online).

Regarding the privacy of training samples, Table IV shows the defense effectiveness of HierarchicalDP against membership inference attacks on different models in the MNIST dataset classification task. With the model ACC impact controlled within 2.4%, HierarchicalDP effectively defends against membership inference attacks. On average, the MPE attack reduced the MIR by 11.7%, and the EMI attack reduced the MIR by

13.5%. Table V shows the defense effectiveness of HierarchicalDP against membership inference attacks on different models in the CIFAR-10 dataset classification task. With the model ACC impact controlled within 4.5%, HierarchicalDP shows slight defense effectiveness against membership inference attacks. On average, the MPE attack reduced the MIR by 11.0%, and the EMI attack reduced the MIR by 2.6%. This is due to the structured characteristics of image samples in the MNIST dataset resulting from central processing, while the CIFAR-10 samples lack central processing.

Regarding the robustness of testing samples, Table VI shows the defense effectiveness of HierarchicalDP against adversarial attacks on different models in the MNIST dataset classification task. With the model ACC impact controlled within 2.7%, HierarchicalDP effectively defends against adversarial attacks. On average, the NIFGSM attack reduced the AER by 59.7%, and the APGD attack reduced the AER by 65.7%. Table VII shows the defense effectiveness of HierarchicalDP against adversarial attacks on different models in the CIFAR-10 dataset classification task. With the model ACC impact controlled within 5.8%, HierarchicalDP shows little defense effectiveness against adversarial attacks. On average, the NIFGSM attack reduced the AER by 9.0%, and the APGD attack reduced the AER by 10.1%. This indicates

TABLE III
EXTENDING EXPERIMENTS DATASETS AND BASELINE MODELS

Symbol	Feature Size	Training Set Size	Testing Set Size	Target Labels	Model Architecture (Baseline Accuracy)
MNIST	784	6k	3k	10	MLP (0.746), DNN (0.760), CNN (0.808)
CIFAR-10	1024×3	10k	10k	10	MLP (0.425), DNN (0.379), CNN (0.608)

TABLE IV
DEFENSE EFFECT OF HIERARCHICALDP AGAINST MEMBERSHIP
INFERENCE ON THE MNIST DATASET

Model	MIR of MPE [30]		MIR of EMI [31]		ACC of Model	
	w/o	w/	w/o	w/	w/o	w/
MLP	0.702	0.599	0.700	0.503	0.746	0.739
DNN	0.616	0.517	0.620	0.500	0.760	0.736
CNN	0.599	0.507	0.613	0.524	0.808	0.803

TABLE V
DEFENSE EFFECT OF HIERARCHICALDP AGAINST MEMBERSHIP
INFERENCE ON THE CIFAR-10 DATASET

Model	MIR of MPE [30]		MIR of EMI [31]		ACC of Model	
	w/o	w/	w/o	w/	w/o	w/
MLP	0.644	0.602	0.633	0.600	0.425	0.417
DNN	0.807	0.756	0.794	0.741	0.379	0.362
CNN	0.737	0.501	0.736	0.743	0.608	0.563

TABLE VI
DEFENSE EFFECT OF HIERARCHICALDP AGAINST ADVERSARIAL ATTACKS
ON THE MNIST DATASET

Model	AER of NIFGSM [32]		AER of APGD [33]		ACC of Model	
	w/o	w/	w/o	w/	w/o	w/
MLP	0.913	0.315	0.925	0.289	0.746	0.743
DNN	0.930	0.328	0.914	0.262	0.760	0.753
CNN	0.893	0.301	0.974	0.292	0.808	0.781

TABLE VII
DEFENSE EFFECT OF HIERARCHICALDP AGAINST ADVERSARIAL ATTACKS
ON THE CIFAR-10 DATASET

Model	AER of NIFGSM [32]		AER of APGD [33]		ACC of Model	
	w/o	w/	w/o	w/	w/o	w/
MLP	0.802	0.695	0.824	0.719	0.425	0.398
DNN	0.891	0.776	0.901	0.894	0.379	0.377
CNN	0.857	0.808	0.891	0.701	0.608	0.550

that HierarchicalDP is only suitable for protecting structured sample classifiers.

In summary, HierarchicalDP provides strong privacy protection and robustness improvement for structured data. As network traffic datasets typically exhibit highly structured characteristics, HierarchicalDP is well suited to enhance the privacy and robustness of these datasets. It also offers protection for centralized image datasets.

VI. CONCLUSION AND FUTURE WORK

In this article, we propose the feature hierarchization strategy and HierarchicalDP to ensure the differential privacy of ML-NIDS while improving their adversarial robustness. HierarchicalDP is a differential privacy technique that protects network traffic sample classifiers, including packet samples and session samples. Compared to the state-of-the-art existing methods, HierarchicalDP reduces the MIR by 9.2% and lowers the AER by 23.3% while keeping the impact on ACC within 4.2%.

Another essential advantage of HierarchicalDP is decoupling. Compared to the differential privacy mechanism that adds noise to the gradient, HierarchicalDP only adds noise to the sample features and does not require modification to the model. In addition to the network traffic classifier, HierarchicalDP can also be used for other structured sample classifiers, such as center-aligned image samples. Future research will focus on utilizing graph convolutional neural networks to discover more accurate relationships between features, aiming to enhance the defensive performance of HierarchicalDP. This approach is intended to be applicable in fields with unstructured samples, such as complex images.

We focus the safety of machine learning on sample data and seek to avoid machine learning models that are currently uninterpretable. Only the bond of feature weights is left between the data and the model. On the one hand, we unify differential privacy and adversarial robustness in machine learning through a formal description that will inspire cross collaboration among relevant researchers. On the other hand, machine learning is essentially the fitting of model distributions to data distributions, and security issues arise in both distributions. This article focuses on addressing privacy and robustness problems through data distribution. However, the above issues can also be addressed through model distributions. It relies on developing interpretable machine learning and will be the focus of future work.

REFERENCES

- [1] I. H. Sarker, "Machine learning: Algorithms, real-world applications and research directions," *SN Comput. Sci.*, vol. 2, no. 3, pp. 1–21, 2021.
- [2] H. Peng, S. Bao, and L. Li, "A survey of security protection methods for deep learning model," *IEEE Trans. Artif. Intell.*, early access, Sep. 12, 2023.
- [3] B. Liu, M. Ding, S. Shaham, W. Rahayu, F. Farokhi, and Z. Lin, "When machine learning meets privacy: A survey and outlook," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–36, 2021.
- [4] K. A. Crockett, L. Gerber, A. Latham, and E. Colyer, "Building trustworthy AI solutions: A case for practical solutions for small businesses," *IEEE Trans. Artif. Intell.*, vol. 4, no. 4, pp. 778–791, Aug. 2023.
- [5] J. Curzon, T. A. Kosa, R. Akalu, and K. El-Khatib, "Privacy and artificial intelligence," *IEEE Trans. Artif. Intell.*, vol. 2, no. 2, pp. 96–108, Apr. 2021.

- [6] C. Qin et al., "Feature fusion based adversarial example detection against second-round adversarial attacks," *IEEE Trans. Artif. Intell.*, vol. 4, no. 5, pp. 1029–1040, Oct. 2023.
- [7] I. Ilahi et al., "Challenges and countermeasures for adversarial attacks on deep reinforcement learning," *IEEE Trans. Artif. Intell.*, vol. 3, no. 2, pp. 90–109, Apr. 2022.
- [8] T. Zhu, D. Ye, W. Wang, W. Zhou, and P. Yu, "More than privacy: Applying differential privacy in key areas of artificial intelligence," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 6, pp. 2824–2843, Jun. 2022.
- [9] A. Triastcyn and B. Faltings, "Bayesian differential privacy for machine learning," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 9583–9592.
- [10] S. Sajadmanesh and D. Gatica-Perez, "Locally private graph neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2130–2145.
- [11] R. Shao, P. Perera, P. C. Yuen, and V. M. Patel, "Open-set adversarial defense with clean-adversarial mutual learning," *Int. J. Comput. Vis.*, vol. 130, no. 4, pp. 1070–1087, 2022.
- [12] A. M. Sadeghzadeh, S. Shiravi, and R. Jalili, "Adversarial network traffic: Towards evaluating the robustness of deep-learning-based network traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1962–1976, Jun. 2021.
- [13] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. Theory Cryptogr. Conf.*, New York, NY, USA: Springer-Verlag, 2006, pp. 265–284.
- [14] K. Chatzikokolakis, M. E. Andrés, N. E. Bordenabe, and C. Palamidessi, "Broadening the scope of differential privacy using metrics," in *Proc. Int. Symp. Privacy Enhancing Technol. Symp.*, pp. 82–102, New York, NY, USA: Springer-Verlag, 2013.
- [15] I. Mironov, "Rényi differential privacy," in *Proc. IEEE 30th Comput. Secur. Found. Symp. (CSF)*, Piscataway, NJ, USA: IEEE Press, 2017, pp. 263–275.
- [16] C. Szegedy et al., "Intriguing properties of neural networks," in *Proc. 2nd Int. Conf. Learn. Representations (ICLR)*, 2014.
- [17] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [18] D. Diochnos, S. Mahloujifar, and M. Mahmoody, "Adversarial risk and robustness: General definitions and implications for the uniform distribution," in *Proc. Adv. Neural Inf. Process. Syst.*, 31, 2018, pp. 10380–10389.
- [19] R. Pinot, F. Yger, C. Gouy-Pailler, and J. Atif, "A unified view on differential privacy and robustness to adversarial examples," in *Proc. Workshop Mach. Learn. CyberSecurity (ECMLPKDD)*, 2019, pp. 368–373.
- [20] D. Bahdanau, K. H. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. 3rd Int. Conf. Learn. Representations (ICLR)*, 2015, pp. 1–15.
- [21] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 6000–6010.
- [22] A. Bommert, X. Sun, B. Bischl, J. Rahnenführer, and M. Lang, "Benchmark for filter methods for feature selection in high-dimensional classification data," *Comput. Statist. Data Anal.*, vol. 143, 2020, Art. no. 106839.
- [23] M. Lecuyer, V. Atlidakis, R. Geambasu, D. Hsu, and S. Jana, "Certified robustness to adversarial examples with differential privacy," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 656–672.
- [24] H. Phan, M. T. Thai, H. Hu, R. Jin, T. Sun, and D. Dou, "Scalable differential privacy with certified robustness in adversarial learning," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 7683–7694.
- [25] B. Skrlj, S. Dzeroski, N. Lavrac, and M. Petkovic, "Feature importance estimation with self-attention networks," 2020, *arXiv:2002.04464*.
- [26] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Piscataway, NJ, USA: IEEE Press, 2017, pp. 712–717.
- [27] D. Stiawan, M. Y. B. Idris, A. M. Bamhdi, and R. Budiarto, "CICIDS-2017 dataset feature analysis with information gain for anomaly detection," *IEEE Access*, vol. 8, pp. 132911–132921, 2020.
- [28] G. Liu, W. Zhang, X. Li, K. Fan, and S. Yu, "VulnerGAN: A backdoor attack through vulnerability amplification against machine learning-based network intrusion detection systems," *Sci. China Inf. Sci.*, vol. 65, no. 7, pp. 28–46, 2022.
- [29] R.-H. Hwang, M.-C. Peng, V.-L. Nguyen, and Y.-L. Chang, "An LSTM-based deep learning approach for classifying malicious traffic at the packet level," *Appl. Sci.*, vol. 9, no. 16, pp. 3414–3428, 2019.
- [30] L. Song and P. Mittal, "Systematic evaluation of privacy risks of machine learning models," in *Proc. 30th USENIX Secur. Symp.*, vol. 1, 2021, pp. 2615–2632.
- [31] J. Ye, A. Maddi, S. K. Murakonda, V. Bindschaedler, and R. Shokri, "Enhanced membership inference attacks against machine learning models," in *Proc. 2022 ACM SIGSAC Conf. Comput. Commun. Secur.*, pp. 3093–3106, 2022.
- [32] J. Lin, C. Song, K. He, L. Wang, and J. E. Hopcroft, "Nesterov accelerated gradient and scale invariance for adversarial attacks," 2019, *arXiv:1908.06281*.
- [33] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 2206–2216.
- [34] M. Nasr, R. Shokri, and A. Houmansadr, "Machine learning with membership privacy using adversarial regularization," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 634–646.
- [35] J. Jia, A. Salem, M. Backes, Y. Zhang, and N. Z. Gong, "MemGuard: Defending against black-box membership inference attacks via adversarial examples," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 259–274.
- [36] H. Salman, M. Sun, G. Yang, A. Kapoor, and J. Kolter, "Denoised smoothing: A provable defense for pretrained classifiers," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 21945–21957.
- [37] M. Alfara, J. C. Pérez, A. Thabet, A. Bibi, P. H. Torr, and B. Ghanem, "Combating adversaries with anti-adversaries," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, 2022, pp. 5992–6000.



Guangrui Liu received the B.E. degree in information security from the School of Computer Science and Technology, China University of Mining and Technology, Xuzhou, Jiangsu, China, in 2018. He is currently working toward the Ph.D. degree in cyberspace security with the School of Cyberspace Science, Harbin Institute of Technology, Harbin, Heilongjiang, China.

His main research interests include artificial intelligence security and network traffic analysis.



Weizhe Zhang (Senior Member, IEEE) received the B.Eng., M.Eng., and Ph.D. degrees in engineering of computer science and technology from Harbin Institute of Technology, Harbin, Heilongjiang, China, in 1999, 2001, and 2006, respectively.

He is currently a Professor with the School of Computer Science and Technology, Harbin Institute of Technology, and the Director of the Cyberspace Security Research Center, Peng Cheng Laboratory, Shenzhen, China. His research interests primarily include parallel computing, distributed computing,

cloud and grid computing, and computer network. He has published more than 100 academic papers in journals, books, and conference proceedings.



Xurun Wang received the B.E. degree in information security from the School of Cyberspace Science, Harbin Institute of Technology, Harbin, Heilongjiang, China, in 2022, where he is currently working toward the Ph.D. degree in Cyberspace Security.

His main research interests include artificial intelligence security and network traffic analysis.



Stephen King is currently working toward the graduate degree with Leland High School, San Jose, CA, USA.



Shui Yu (Fellow, IEEE) obtained his Ph.D. degree in computer science from Deakin University, Melbourne, Australia, in 2004.

He is a Professor with the School of Computer Science and the Deputy Chair of University Research Committee, University of Technology Sydney, Ultimo, NSW, Australia. His research interests include cybersecurity, network science, big data, and mathematical modeling. He has published five monographs and edited two books and published more than 500 technical papers

at different venues, such as IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING, IEEE/ACM TRANSACTIONS ON NETWORKING, and IEEE INFOCOM. His current h-index is 72. He promoted the research field of networking for big data since 2013, and his research outputs have been widely adopted by industrial systems, such as Amazon cloud security.

Prof. Yu is currently serving on the editorial boards as an Area Editor of IEEE COMMUNICATIONS SURVEYS AND TUTORIALS and an Editor of IEEE INTERNET OF THINGS JOURNAL. He served as a Distinguished Lecturer of IEEE Communications Society (2018–2021). He is a Distinguished Visitor of IEEE Computer Society, and an Elected Member of Board of Governors of IEEE VTS and ComSoc, respectively. He is a member of ACM and AAAS.